



Universidad Anáhuac Oaxaca
Facultad de Ingeniería

Modernización del Robot Move Master EX

*Implementación de un nuevo controlador
y su electrónica asociada*

Alumna:

Daniela Tobón Pérez

Catedrático:

M.C. Juan Pablo Avendaño Santiago

Materia:

Practicum 2

Periodo:

Enero – Mayo 2026

Oaxaca de Juárez, Oaxaca.

Índice general

1. Introducción	3
2. Palabras Claves	5
3. Rediseño e Implementación de la Etapa de Potencia	6
3.1. Lista de materiales	6
3.2. Antecedentes: El Sistema de Control Original	7
3.3. Análisis de Requerimientos Energéticos	10
3.4. Gestión de Voltaje: El uso de 12V en Motores de 24V	11
3.5. Implementación de la Fuente Conmutada	12
3.6. Placa de Interconexión Rápida	14
3.7. Diseño del Enclosure y Seguridad	16
4. Interfaz de Comunicación y Lógica de Control en LabVIEW	18
4.1. Especificaciones Técnicas del Controlador: ATmega2560	19
4.2. Protocolo de Comunicación: Estructura y Fiabilidad	20
4.3. Definición del Protocolo de Comunicación	22

4.4. Implementación de Estructuras de Datos en C	23
4.5. Implementación de Estructuras de Datos en C	23
4.6. Definición de Enumeraciones (Enums)	24
4.7. Restricciones de Hardware y Determinismo	25
4.8. Máquina de Estados de Control de Movimiento	26
4.9. Adquisición de Datos y Gestión de Periféricos I/O	26
4.10. Interfaz de Usuario en LabVIEW	30
Bibliografía	33

Índice de figuras

3.1. Transformador original	7
3.2. Especificaciones del transformador	7
3.3. Componentes de la Drive Unit	8
3.4. Panel del controlador viejo	9
3.5. Tabla de especificaciones del motor y frenos. [6]	10
3.6. Fuente de poder conmutada seleccionada	13
3.7. Nuevo regulador de voltaje para los microcontroladores	14
3.8. Ejemplos de barras de buses usadas en gabinetes	15
3.9. Creación de una placa de distribución de potencia simple	15
3.10. Conexión de la fuente de poder al regulador de voltaje y barra de bus.	16
3.11. Tabla con valores AWG	16
4.1. Topología.	21

4.2. Dashboard.	31
4.3. Ventana de control de la interfaz.	31
4.4. Ventana de PID de la interfaz.	32

Introducción

El desarrollo de este brazo robótico de 6 grados de libertad (GDL) representa un reto que integra la parte mecánica, la electrónica de potencia y la programación. En un sistema de este tipo, no basta con lograr el movimiento de los actuadores, sino que es fundamental garantizar que el equipo sea capaz de autoprotegerse ante eventos críticos, como colisiones o sobrecargas de corriente. Por esta razón, el proyecto se enfoca en implementar un sistema de monitoreo que asegure la integridad de los motores y de toda la etapa de control involucrada.

Para la gestión de los datos, se diseñó una interfaz en LabVIEW utilizando una arquitectura de *Queue Message Handler* (QMH). Este enfoque permite que el programa procese de manera simultánea la lectura de los sensores ACS712 y la comunicación con el Arduino Mega sin comprometer la fluidez de la interfaz. De esta manera, es posible visualizar en tiempo real los picos de corriente durante el arranque o detectar si un motor se encuentra bajo un esfuerzo excesivo, lo que permite interrumpir la alimentación antes de que ocurra un daño en los *drivers* o en la fuente de poder.

Finalmente, se dio prioridad al diseño y construcción de un gabinete de control (prototipo) utilizando el software *SolidWorks*. La realización de estos primeros prototipos en una etapa temprana del desarrollo permite identificar carencias en el diseño y mejorar los siguientes intentos antes de la fabricación final. Este chasis permitió validar la distribución de los componentes, como los controladores BTS7960 y el sistema de ventilación, además de evitar la necesidad de conectar y desconectar componentes constantemente. Gracias a este enfoque de mejora continua, el proceso de prueba del robot resulta mucho más ordenado, seguro y eficiente.

Palabras Claves

Gabinete de control, Prototipado rápido, Gestión térmica, Distribución modular, Interfaz de potencia, SolidWorks, Brazo robótico.

Rediseño e Implementación de la Etapa de Potencia

Resumen del Capítulo

Este capítulo detalla la transición del sistema de alimentación original hacia una arquitectura moderna de potencia. Se analizan los requerimientos de corriente de los actuadores, la integración de fuentes conmutadas y las soluciones de hardware personalizadas para optimizar la distribución eléctrica y la seguridad del robot.

3.1. Lista de materiales

- Fuente de poder de 12V y con capacidad de 40 Amp.

- Regulador de voltaje Buck de salida de 1.1v a 40V.
- Placa de suministro de voltaje (barra de bus).
- Cables de calibre 16 y 20 AWG.
- Multímetro.

3.2. Antecedentes: El Sistema de Control Original

Se realiza una revisión del controlador heredado del *Move Master EX*, analizando las limitaciones de su fuente de voltaje interna y la necesidad de una etapa de potencia independiente que permita una mayor flexibilidad y protección de los componentes electrónicos modernos. De acuerdo con las especificaciones técnicas detalladas en el manual del fabricante [5], el sistema original presentaba una unidad de control integral que centralizaba múltiples funciones en un solo chasis. Según las especificaciones de placa y manual, el equipo estaba diseñado para operar con un rango de alimentación versátil de **120V / 220V / 230V / 240V AC $\pm 10\%$** , con una potencia aparente nominal de **0.5 KVA**. Esta flexibilidad permitía al robot adaptarse a las diferentes líneas de voltaje según el país de operación, pero implicaba una etapa de transformación interna voluminosa y pesada.



corto para la lista]hi

Figura 3.1: Transformador de la etapa original de potencia del controlador.

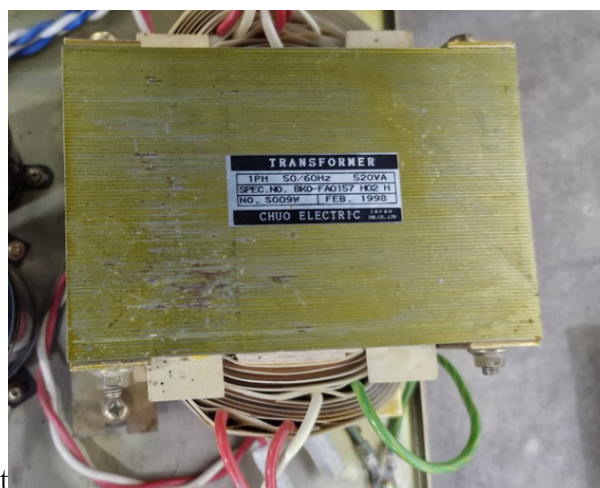


Figura 3.2: Especificaciones del transformador: 1 PH, 50/60 Hz y 520 VA.

3.2.1. Funcionalidades de la Drive Unit

Es importante destacar que el controlador no funcionaba únicamente como una fuente de poder; operaba como una *Drive Unit* compleja que gestionaba diversos subsistemas:

- **Interfaz de Comunicación:** Incorporaba un puerto serial **RS-232** para la vinculación con computadoras de la época.
- **Control Manual:** Interfaz dedicada para la *Teaching Box*, permitiendo el movimiento punto a punto.
- **Gestión de Actuadores:** Salidas específicas para *Motor Power* y recepción de señales a través de *Motor Signal*.
- **Herramientas Finales:** Opciones de configuración para el uso de terminales o efectores accionados tanto por DC como por AC.

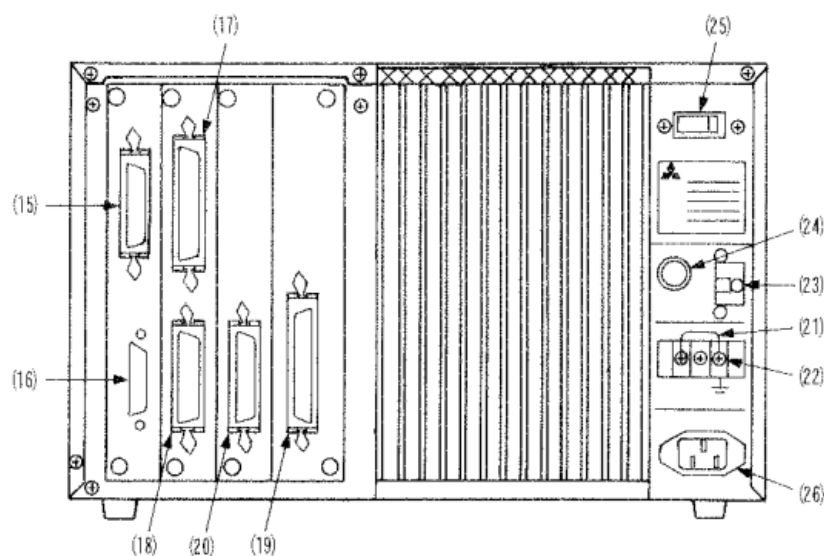


Figura 3.3: Componentes de la Drive Unit. Contiene un conector central para la conexión con la computadora, la teaching box, la alimentación de los motores, señales de control para el motor, selección del tipo de voltaje (AC/DC) del efector, etc. [5]

3.2.2. Justificación del Rediseño y Simplificación

Al realizar la apertura del chasis, se identificó una arquitectura basada en múltiples paneles de circuitos integrados densamente poblados que, por su naturaleza analógica y lógica discreta, presentan una carga parasitaria considerable, generando calor y consumo de voltaje de forma constante. Fig. 3.4.

Dado que el objetivo de este proyecto es el desarrollo de algoritmos y programas personalizados para controlar los motores de manera específica, la mayoría de las funciones predefinidas del controlador original (como el protocolo RS-232 propietario o la lógica de la *Teaching Box*) resultan redundantes.

Este análisis justifica la necesidad de migrar hacia una fuente conmutada moderna, eliminando la electrónica obsoleta que genera un gasto energético innecesario. Esta transición permite una gestión de potencia directa, compacta y eficiente, centrada exclusivamente en alimentar los nuevos drivers y el microcontrolador que ejecutará el software diseñado a medida.

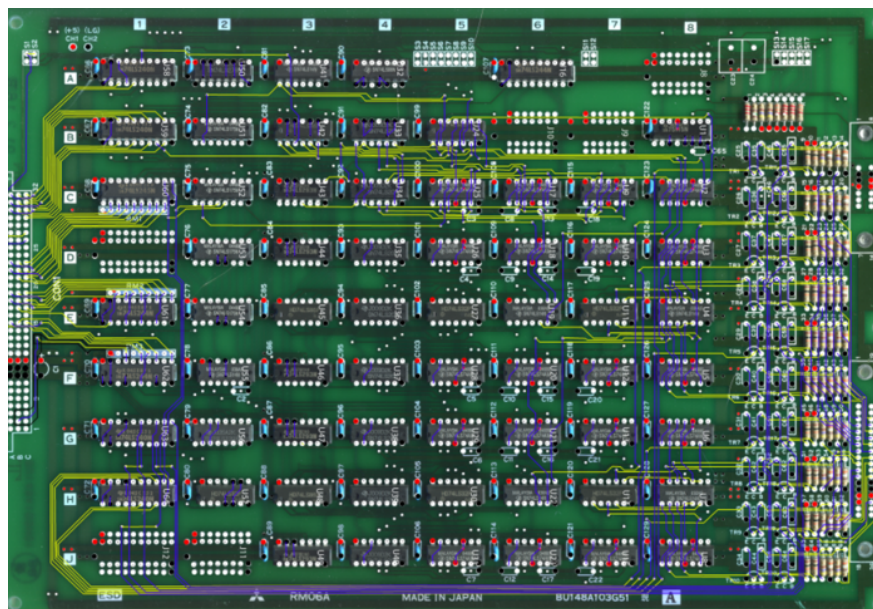


Figura 3.4: Vista interna de uno de los paneles electrónicos del controlador original. Se observa la densidad de circuitos integrados que conforman la lógica de control y la unidad de potencia del sistema. [7].

3.3. Análisis de Requerimientos Energéticos

Para determinar la capacidad necesaria de la fuente de alimentación, es fundamental analizar el consumo de corriente de todos los actuadores en su estado más crítico. Como se observa en la Figura 3.5, la cual presenta datos técnicos recopilados en la investigación de Sebastián, C. M. J., y Sebastián, M. C. J. [6], se detallan las especificaciones nominales de los motores y frenos del brazo robótico.

Especificaciones	Motor J1	Motor J2	Motor J3	Motor J4	Motor J5	Motor Pinza	Frenos J2 y J3
Marca	DTME-2730	DTME-2730	DTME-2730	DTME-2730	DTME-2730	DTME-2730	Type BBS-41-1.5
Voltaje	24 V	24 V	24 V	24 V	24 V	12 V	12 V
Potencia	30W	30W	30W	11W	11W	11 W	3.6 W
Corriente sin carga	1.4 A	2.35 A	1.7 A	1.7 A	2.5 A	200 mA	N/A
Corriente Con carga	2 A	2.75 A	2.2 A	2 A	2.65 A	200 mA	N/A

Figura 3.5: Tabla de especificaciones del motor y frenos. [6]

Considerando un escenario de máxima exigencia operativa, donde todos los ejes se desplazan simultáneamente bajo carga, la corriente total requerida por los actuadores se obtiene mediante la sumatoria de las intensidades máximas registradas en la columna de “Corriente con carga”:

$$I_{motores} = 2,0A + 2,75A + 2,2A + 2,0A + 2,65A + 0,2A = 11,8 \text{ A}$$

Además del consumo de los motores, el sistema integra una etapa lógica y de potencia que debe ser considerada en el presupuesto energético total:

- **Microcontroladores:** Un Arduino Mega y dos módulos ESP32, encargados de la gestión de trayectorias y comunicación inalámbrica.
- **Drivers de Potencia:** Seis puentes H modelo **BTS7960**.
- **Sistemas de Frenado:** Frenos electromagnéticos que operan a 12V.

Bajo estas consideraciones, se determinó que la fuente de alimentación debe ser capaz de suministrar de forma constante entre **13A y 14A**. Este margen de seguridad es vital para absorber los picos de corriente transitorios durante el arranque de los motores y para garantizar que el voltaje de 5V (obtenido mediante un regulador *step-down* desde la línea de 12V) permanezca libre de ruido y caídas que pudieran provocar reinicios en los microcontroladores.

3.4. Gestión de Voltaje: El uso de 12V en Motores de 24V

La selección del voltaje de operación para los actuadores del robot no es una decisión arbitraria, sino el resultado de una fase de experimentación previa y un análisis de eficiencia energética. Durante el semestre anterior, se realizaron pruebas preliminares operando los motores a un nivel de 12V DC, obteniendo resultados satisfactorios en cuanto a estabilidad y control de movimiento.

Al profundizar en la investigación técnica, se determinó que operar a 12V representa un balance óptimo para los objetivos de este proyecto. Según la literatura técnica sobre máquinas eléctricas, existe una relación directa entre el voltaje aplicado y las características de salida del motor:

“En los motores de corriente continua, el torque es proporcional a la corriente, mientras que la velocidad es proporcional al voltaje aplicado. Reducir el voltaje de operación permite un control más granular y un menor consumo de potencia base, aunque esto implique una reducción proporcional en el torque máximo y la velocidad final” [3].

Para la aplicación actual del *Move Master EX* en un entorno de laboratorio, no se requiere que el brazo manipule cargas máximas de diseño. Por lo tanto, el sacrificio en el torque es una concesión aceptable que ofrece las siguientes ventajas:

- **Eficiencia Térmica:** Menor generación de calor en los drivers y en los propios motores, prolongando la vida útil de los componentes.

- **Seguridad de Implementación:** Un sistema que opera a 12V reduce los riesgos de picos de corriente severos durante las fases de programación y testeado de algoritmos.
- **Simplificación de la Etapa de Potencia:** Para el funcionamiento integral del sistema, se identificaron tres niveles de tensión críticos: los motores operan nominalmente a **24V**, los sistemas de frenado requieren **12V**, y la lógica de control (microprocesadores y sensores) funciona a **5V**.

El uso de motores de 24V sugería la implementación de una fuente de dicha magnitud; sin embargo, esto habría implicado la necesidad de integrar una fuente conmutada adicional de 12V para los frenos, aumentando el volumen y la complejidad del sistema. Basado en la experimentación del semestre anterior, se determinó que operar los motores a 12V no resulta ser una mala decisión de ingeniería si se prioriza el consumo energético sobre el torque máximo

3.5. Implementación de la Fuente Conmutada

La elección de una fuente de alimentación conmutada (*Switching Power Supply*) sobre un sistema lineal tradicional se basa en su alta eficiencia, dimensiones compactas y estabilidad bajo cargas variables. Para este proyecto, se seleccionó una unidad comercial con chasis de aluminio, material que actúa como un disipador de calor pasivo eficiente, garantizando la integridad de los componentes internos durante periodos prolongados de operación.

Las especificaciones técnicas de la fuente seleccionada son las siguientes:

- **Capacidad de Salida:** Suministra una tensión constante de **12V DC** con una intensidad de hasta **40A**, lo cual cubre con amplitud los 14A calculados como requerimiento máximo, permitiendo que la fuente trabaje de manera holgada y sin sobrecalentamientos.
- **Sistemas de Seguridad Integrados:** Esta unidad cuenta con protección activa contra **sobrecarga**, **cortocircuito** y **sobretensión**, desconectando la salida automáticamente ante cualquier anomalía eléctrica para proteger los drivers BTS7960 y la lógica de control.
- **Protección Física:** El diseño incluye fusibles de protección internos y una rejilla de ventilación en la caja de aluminio que facilita el flujo de aire, crucial para mantener la

estabilidad térmica del sistema.

Esta sobredimensión en el amperaje (40A frente a los 14A requeridos) se decidió estratégicamente para permitir futuras expansiones del sistema, como la adición de actuadores adicionales asegurando que la fuente nunca sea un cuello de botella en el rendimiento del robot. Además de que el precio-capacidad era bueno.



Figura 3.6: Unidad de potencia seleccionada para el sistema. El diseño industrial en aluminio y su capacidad de 40A garantizan un suministro estable y seguro para los motores, frenos y la etapa lógica.

3.5.1. Implementación del Módulo Step-Down para la Etapa Lógica

Debido a que la fuente conmutada principal entrega 12V y la lógica de control (Arduino Mega y ESP32) requiere una tensión estable de 5V (o de 7V en el caso de ser alimentados por medio del pin de Vin del Arduino), se integró un regulador de voltaje de tipo *Step-Down* (convertidor Buck) de alta potencia.

Las especificaciones técnicas del módulo seleccionado justifican su uso en este entorno industrial:

- **Rangos de Operación:** El módulo admite un voltaje de entrada de **6V a 40V** y permite un ajuste preciso de salida entre **1.2V y 36V**, lo que lo hace ideal para reducir los 12V de la fuente principal a los 5V requeridos.

- **Eficiencia y Potencia:** Cuenta con una eficiencia de conversión del **97 %**, minimizando el desperdicio energético en forma de calor. Además, es capaz de suministrar hasta **20A** de salida, garantizando que los tres microcontroladores y sensores operen con un flujo de corriente sobradamente estable.
- **Gestión Térmica y Seguridad:** El dispositivo incorpora grandes disipadores de calor de aluminio y protección activa contra **cortocircuito**, añadiendo una capa de seguridad crítica antes de llegar a la electrónica de control.
- **Controles de Precisión:** Incluye dos potenciómetros de ajuste: **CV** (*Constant Voltage*) para fijar el voltaje de salida y **CC** (*Constant Current*) para limitar la corriente de salida si fuera necesario.

Esta configuración permite que la etapa lógica esté eléctricamente aislada de las fluctuaciones de consumo de los motores, utilizando el regulador como una barrera de protección y estabilidad.

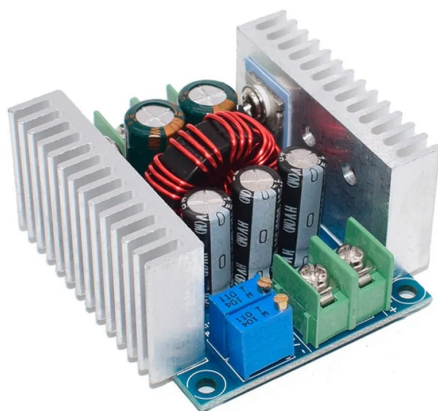


Figura 3.7: Regulador de voltaje salida de 1.1v a 40v, 20a y 300w. Utilizado para la parte lógica del brazo robótico.

3.6. Placa de Interconexión Rápida

Debido al elevado flujo de corriente requerido para alimentar simultáneamente los seis drivers BTS7960, se diseñó y fabricó una PCB que funcionara como una **barra de bus** que se usan en gabinetes de motores. El propósito de esta placa es organizar la entrega de energía desde

la fuente de 12V hacia cada controlador de manera segura y eficiente, evitando la caída de tensión por conexiones deficientes.



Figura 3.8: Barras de buses usadas en gabinetes de motores y otros sistemas de distribución.

La placa integra dos series de borneras de alta resistencia para facilitar la conexión y desconexión de los módulos. Un aspecto crítico en su diseño fue el grosor de las pistas de cobre pues es necesario considerar el tamaño de las pistas debido a la resistencia hacia estos amperajes.



Figura 3.9: Placa de distribución de potencia diseñada para la administración de 12V hacia los drivers de los motores.

3.6.1. Dimensionamiento del Cableado

Para garantizar la seguridad del sistema y evitar caídas de tensión por resistencia, se seleccionó un conductor de calibre 16 AWG para la conexión principal entre la fuente de poder y la placa de distribución, ya que este calibre soporta con margen de seguridad los 14 A requeridos por el sistema completo. Por otro lado, para el cableado que va desde los drivers hacia cada motor, se optó por un calibre 20 AWG, el cual excede el requerimiento mínimo

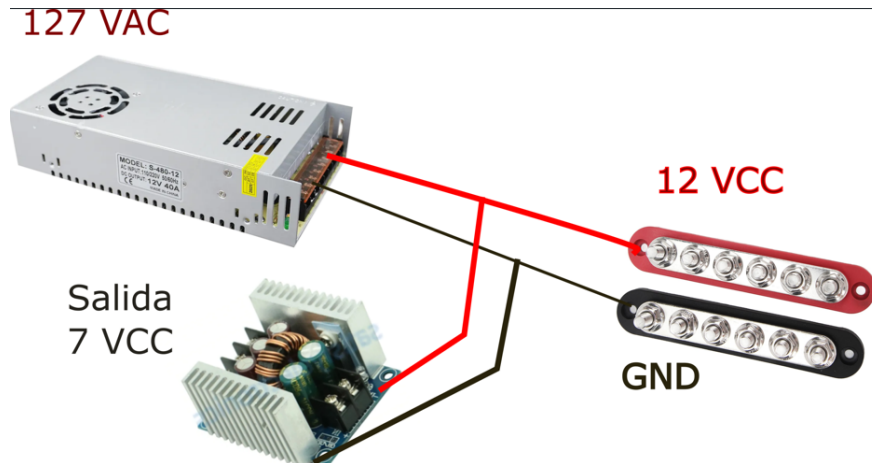


Figura 3.10: Conexión de la fuente de poder al regulador de voltaje y barra de bus.

de 3 A, proporcionando la robustez necesaria para soportar los picos de corriente durante el arranque sin comprometer la flexibilidad del cableado en las articulaciones del robot.

Temperatura	60°C
Calibre del cable	Amperaje soportado
20 AWG	2 A
18 AWG	10 A
16 AWG	13 A
14 AWG	18 A
12 AWG	25 A

Figura 3.11: Tabla donde se asocian los valores del calibre del cable al amperaje.

3.7. Diseño del Enclosure y Seguridad

Para el diseño de donde se encuentra centralizada toda la electrónica del brazo se priorizó la protección física y térmica de los componentes; así como, la capacidad de poder sostener el brazo robótico sobre una de sus caras. El diseño es discutido en otros capítulos, pero tras la selección de los componentes dentro de este capítulo, se considera necesario un flujo de aire

constante, así como la posibilidad de poder tener un acceso estratégico a los componentes para monitorear las conexiones y poder cambiar componentes de manera modular.

Resultados y Validación de Fase

En esta fase se logró la transición exitosa de un sistema de alimentación rudimentario a una infraestructura de potencia industrial. Los resultados clave se resumen a continuación:

- **Gestión de Voltaje y Protección de Actuadores:** Se determinó el uso de **12V** como tensión de operación para los motores. Esta decisión evitó picos de corriente severos que podrían comprometer la integridad de los devanados de los motores originales, garantizando una operación más suave y segura durante las pruebas de movimiento.
- **Modernización de la Fuente de Alimentación:** Se reemplazó el sistema original por una **fente conmutada de 12V a 40A**. Esto no solo reduce drásticamente el volumen del controlador, sino que añade capas críticas de seguridad mediante sistemas integrados contra **sobrecarga, cortocircuito y sobreten-sión**.
- **Estabilidad Lógica mediante Conversión Buck:** Para alimentar la etapa de control, se implementó un regulador *step-down* (Buck) de alta eficiencia.
- **Criterios de Conducción y Visión de Diseño PCB:** Se realizó la selección de calibres de cable adecuados para soportar el flujo de corriente entre la fuente y los drivers BTS7960. Este análisis de amperaje sienta las bases para la siguiente fase de desarrollo, donde estas especificaciones se traducirán en el diseño de PCBs personalizadas, considerando anchos de pista calculados por densidad de corriente y un posicionamiento estratégico de componentes para minimizar el ruido e interferencias.

Interfaz de Comunicación y Lógica de Control en LabVIEW

Resumen del Capítulo

En este capítulo se describe el desarrollo de una interfaz de control robusta y versátil, diseñada bajo un esquema de comunicación serial (UART) que permite la compatibilidad con cualquier microcontrolador que soporte este estándar. A continuación, se detalla la implementación de la lógica interna, la estructuración de datos y el protocolo de comunicación desarrollado para garantizar el determinismo en el control del sistema.

4.1. Especificaciones Técnicas del Controlador: ATMe-ga2560

El núcleo del sistema de control es el microcontrolador de 8 bits **ATMega2560**, basado en la arquitectura RISC avanzada de Harvard. Esta elección técnica se justifica por su capacidad de procesamiento paralelo de instrucciones en un solo ciclo de reloj, optimizando el *throughput* para tareas de control en tiempo real.

4.1.1. Arquitectura y Capacidad de Procesamiento

El procesador opera a una frecuencia de oscilación de **16 MHz** mediante un cristal de cuarzo externo. Esto se traduce en una velocidad de ejecución cercana a los **16 MIPS** (millones de instrucciones por segundo), permitiendo que la mayoría de las instrucciones se completen en un periodo de reloj de tan solo **62.5 ns**. Esta velocidad es crítica para la gestión del lazo de control de 4 ms y la lectura de los encoders de cuadratura, donde la latencia de respuesta define la estabilidad del brazo.

4.1.2. Resolución y Gestión de Señales PWM

Para el accionamiento de los puentes H (BTS7960) que controlan los motores de corriente continua, el Arduino Mega dispone de 15 canales de modulación por ancho de pulso (PWM) generados por hardware.

- **Resolución Temporal:** Por defecto, el sistema opera con una resolución de **8 bits** ($2^8 = 256$ niveles de duty cycle), lo que permite un control granular de la velocidad de los motores.
- **Frecuencias de Conmutación:** Las frecuencias de PWM están determinadas por los divisores de frecuencia (*prescalers*) de los temporizadores internos. En la configuración estándar, los pines 4 y 13 operan a aproximadamente **980 Hz**, mientras que el resto de los pines PWM operan a **490 Hz**. Esta frecuencia es suficiente para minimizar el rizado de corriente en los devanados del motor sin inducir pérdidas excesivas por conmutación en los MOSFETs del driver.

4.1.3. Temporización e Interrupciones (Timers)

La característica más potente del ATmega2560 para este proyecto es su sistema de temporizadores. El microcontrolador integra 6 *timers*: dos de 8 bits y cuatro de **16 bits** (Timer 1, 3, 4 y 5).

- **Timer 5 (16-bit):** Se utiliza específicamente para generar la base de tiempo determinista del sistema. Al operar con una resolución de 16 bits, permite una configuración extremadamente precisa de interrupciones para el conteo de pasos de los encoders y el envío de telemetría, evitando el *jitter* que ocurriría en un ciclo de *polling* convencional.
- **Vectores de Interrupción:** El controlador soporta hasta 8 interrupciones externas (INT0-INT7) y múltiples interrupciones por cambio de pin (PCINT), permitiendo que los flancos de subida y bajada de los encoders de cuadratura activen rutinas de servicio de interrupción (ISR) de baja latencia.

4.1.4. Recursos de Memoria y Comunicación

- **Memoria Flash (256 KB):** Espacio suficiente para almacenar la máquina de estados compleja y las tablas de datos para las rutinas de movimiento.
- **SRAM (8 KB):** Crucial para el manejo de las estructuras (`Motor_t`) y los buffers de comunicación serial sin riesgo de desbordamiento de pila.
- **Interfaces UART:** Cuenta con 4 puertos UART independientes, lo que permite mantener la comunicación con LabVIEW a **500,000 baudios** en el puerto 0, mientras se procesan simultáneamente los datos del Joystick Bluetooth en el puerto 1.

4.2. Protocolo de Comunicación: Estructura y Fiabilidad

Inspirado en el estándar industrial Modbus, se implementó un protocolo de comunicación binario personalizado. La elección de un formato binario sobre uno de texto (como strings de comandos) responde a la necesidad de optimizar el ancho de banda y reducir la latencia de procesamiento.

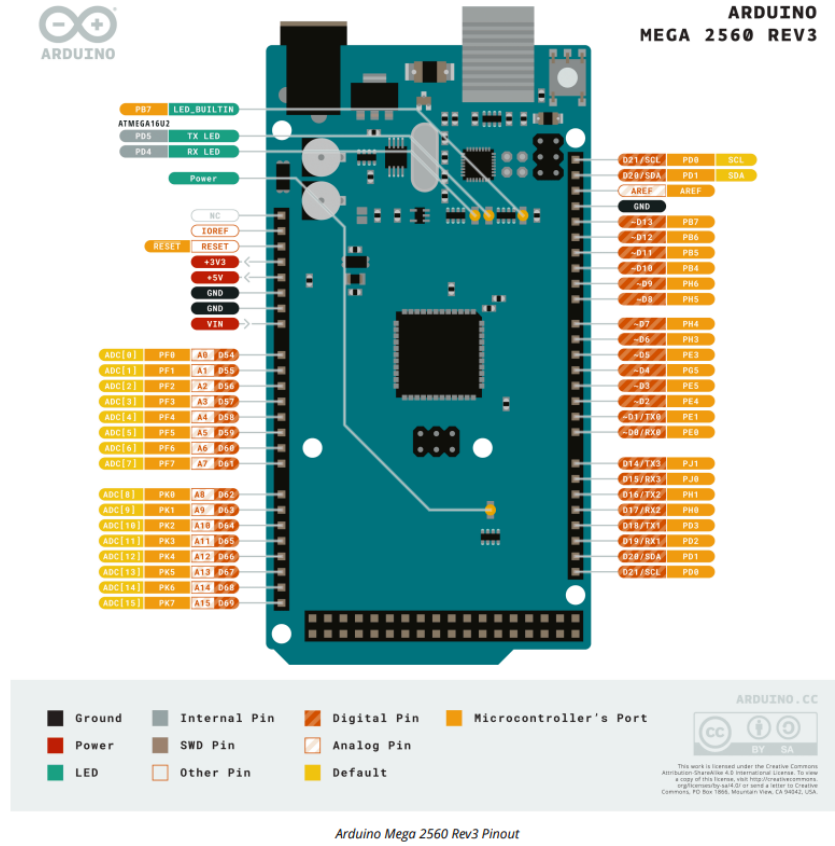


Figura 4.1: Topología del arduino Mega.

La trama de datos consta de un paquete de **31 bytes** diseñado para ser decodificado eficientemente en LabVIEW. Esta estructura asegura que no se pierda la integridad del mensaje mediante los siguientes campos:

- **Cabecera (Header):** Un byte de sincronización (0xAA) que indica el inicio de la trama.
- **Estado de Motores:** Un arreglo de 3 bytes donde se empaquetan los estados lógicos de los 6 motores.
- **Finales de Carrera:** Un byte dedicado a reportar el estado de los sensores de límite, permitiendo una reacción inmediata ante colisiones.
- **Telemetría de Pasos:** 24 bytes que contienen la posición actual (int32_t) de cada uno de los encoders.

- **Ciclo de Redundancia Cíclica (CRC-8):** Un mecanismo de verificación que suma los bytes previos para asegurar que los datos no se corrompieron durante la transmisión.

Esta arquitectura es la mejor opción para el proyecto, ya que permite una reestructuración sencilla en LabVIEW mediante el bloque *Unflatten from String*, manteniendo una comunicación confiable y determinista.

4.3. Definición del Protocolo de Comunicación

La comunicación entre LabVIEW y el Arduino Mega se rige por una trama binaria de longitud fija de 31 bytes. A continuación, se presenta el mapa de memoria de la trama enviada desde el controlador:

Sincronía	ID	Sensores	Estados	Posicionamiento	Verificación
Header	Funct.	Límites	Motores 0-5	Motor 0 ... Motor 5	CRC-8
1 Byte	1 Byte	1 Byte	3 Bytes	24 Bytes (4 por eje)	1 Byte

Tabla 4.1: Estructura macro de la trama de telemetría.

Tabla 4.2: Desglose detallado del paquete de datos (31 Bytes).

Byte(s)	Campo	Tipo de Dato	Descripción Técnica
0	Header	uint8_t	Byte de sincronía constante (0xAA).
1	Función	uint8_t	Identificador del modo de operación activo.
2	Límites	uint8_t	Estado de finales de carrera (Lógica NC).
3 – 5	Edo. Motores	uint8_t[3]	Bit-packing de estados (3 bits/motor).
6 – 9	Pasos M0	int32_t	Posición actual Eje 1 (Base).
10 – 13	Pasos M1	int32_t	Posición actual Eje 2 (Hombro).
14 – 17	Pasos M2	int32_t	Posición actual Eje 3 (Codo).
18 – 21	Pasos M3	int32_t	Posición actual Eje 4 (Pitch).
22 – 25	Pasos M4	int32_t	Posición actual Eje 5 (Roll).
26 – 29	Pasos M5	int32_t	Posición actual Eje 6 (Gripper/Giro).
30	CRC	uint8_t	Checksum de integridad (Suma de bytes).

4.4. Implementación de Estructuras de Datos en C

Para optimizar el firmware, se implementaron estructuras (`struct`) que permiten un manejo eficiente de la memoria y facilitan el mantenimiento del código. En el contexto de un microcontrolador de 8 bits, esta organización es fundamental para el manejo de múltiples periféricos de forma concurrente.

En este proyecto destacan dos estructuras principales:

1. **Motor_t**: Encapsula los parámetros operativos de cada articulación, incluyendo pines de PWM, contadores de encoders, variables de error y estados de reporte. Esto permite una abstracción donde cada eje se gestiona como una entidad independiente.
2. **MsgDatos_t**: Estructura diseñada bajo el concepto de empaquetamiento de memoria, que representa fielmente la trama de 31 bytes. Esto asegura que la disposición de los datos en la SRAM coincida exactamente con la secuencia de transmisión UART.

4.5. Implementación de Estructuras de Datos en C

Para el desarrollo del firmware, se utilizaron estructuras (`struct`) en lenguaje C. El uso de estas permite encapsular variables relacionadas bajo un mismo nombre, facilitando la organización del código y el paso de datos entre funciones de control y comunicación.

Listing 4.1: Estructura de control para articulaciones individuales

```

1  typedef struct {
2      uint8_t pinPWM;           // Pin de control de velocidad
3      uint8_t pinDir;          // Pin de sentido de giro
4      int32_t pasosActuales;    // Contador de pasos (encoder)
5      int32_t pasosMeta;       // Objetivo de posicionamiento
6      uint8_t estadoModo;      // Modo activo (IDLE, FAST, FOLLOW)
7      bool limiteActivado;     // Flag de seguridad (NC)
8  } Motor_t;

```

La segunda estructura fundamental es la que define el paquete de telemetría. Al utilizar el atributo `packed`, aseguramos que no existan espacios de relleno (*padding*) entre los miem-

bros de la estructura, garantizando que el tamaño total sea de exactamente 31 bytes para coincidir con la trama de LabVIEW.

Listing 4.2: Estructura de la trama de telemetría (31 Bytes)

```

1  typedef struct __attribute__((packed)) {
2      uint8_t header;           // Byte de sincronia (0xAA)
3      uint8_t idFuncion;       // Identificador de comando
4      uint8_t limites;         // Bits de sensores de carrera
5      uint8_t estados[3];      // Estados de los 6 motores
6      int32_t posiciones[6];   // Encoders de 32 bits por eje
7      uint8_t crc;             // Checksum de verificacion
8  } MsgDatos_t;

```

4.6. Definición de Enumeraciones (Enums)

Para garantizar la integridad de la máquina de estados y optimizar la comunicación serial, se implementaron enumeraciones que separan la lógica de control interno de los reportes enviados a la interfaz de usuario.

Listing 4.3: Enumeradores para reportes y comportamientos del sistema

```

1  // Reporta a Labview el estado simplificado (Maximo 8 estados)
2  enum EstadoReporte {
3      LV_IDLE = 0,           // 000: Quieto / Esperando
4      LV_MOVIENDO = 1,      // 001: Ejecutando trayectoria o comando manual
5      LV_HOMING = 2,        // 010: En proceso de busqueda de cero
6      LV_ERROR_TRABADO = 3  // 011: Meta no alcanzada / Obstruccion
7  };
8
9  // Define el modo de operacion solicitado por el controlador
10 enum Comportamiento_t {
11     MODE_IDLE = 0,         // Reposo activo
12     MODE_FAST_MOV = 1,     // Movimiento rapido (Jogging)
13     MODE_MOVE_DEG = 2,     // Posicionamiento por grados
14     MODE_FOLLOW_ROUTINE = 3, // Ejecucion de trayectorias (Excel)
15     MODE_HOMING = 4,       // Rutina de referenciado
16     MODE_EMERGENCY_STOP = 5 // Paro de emergencia inmediato
17 };

```

Adicionalmente, se utiliza un enumerador específico para la gestión detallada de la lógica

de control interna. Este permite que cada articulación rastree su progreso durante tareas complejas como el referenciado (*Homing*) o el posicionamiento punto a punto.

Listing 4.4: Enumeradores para la logica interna de control

```

1 // Mantiene la logica completa para el control de pasos interno
2 enum MaqEstado_t {
3     MQ_IDLE = 1,          // Motor en espera
4     MQ_HOMING = 2,        // Ejecutando fase de busqueda
5     MQ_MOVIENDO = 3,      // En transito hacia la meta
6     MQ_META = 4           // Posicion alcanzada con exito
7 };
8
9 // Fases individuales para la rutina de referenciado
10 enum Homing_t {
11     MAQ_HOMING_F1 = 1,    // Busqueda rapida de sensor
12     MAQ_HOMING_F2 = 2,    // Retroceso de precision
13     MAQ_HOMING_F3 = 3     // Ajuste final de cero
14 };

```

Esta distinción es fundamental para el determinismo del sistema. Mientras que la lógica interna utiliza estados detallados para el control de los motores, la telemetría enviada a LabVIEW se compacta en el enumerador EstadoReporte para cumplir con la restricción de 3 bits por motor dentro de la trama de 31 bytes.

4.7. Restricciones de Hardware y Determinismo

La arquitectura basada en el ATmega2560 requiere una gestión estricta de los tiempos de ciclo. El proceso de envío serial, al ser asíncrono y dependiente de un buffer, puede comprometer la estabilidad del sistema si no se regula adecuadamente.

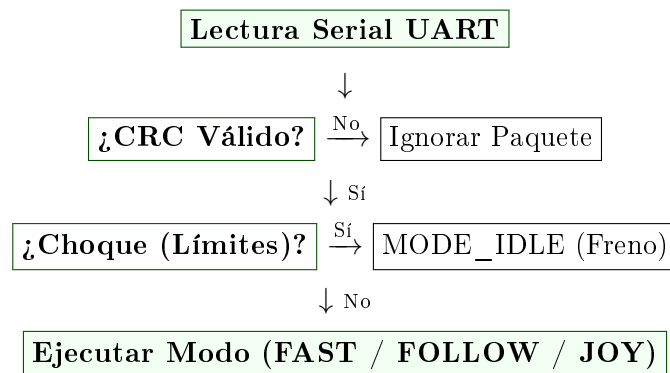
Se estableció un **periodo de control determinista de 4ms (250 Hz)**. La prioridad del procesador se asignó a la actualización de encoders mediante interrupciones y a la lógica de control de los puentes H. El envío de telemetría está sincronizado para ejecutarse al final de cada ciclo, garantizando que el sistema mantenga un tiempo de respuesta constante y predecible bajo cualquier condición de carga.

4.8. Máquina de Estados de Control de Movimiento

La lógica de operación reside en la función `actualizarLogicaMotor`, una máquina de estados finitos (FSM) que gobierna el comportamiento dinámico del brazo Mitsubishi. La transición entre estados depende tanto de los comandos recibidos vía LabVIEW como de las condiciones de seguridad detectadas por hardware.

Los estados operativos definidos son:

- **MODE_IDLE:** Estado de seguridad predeterminado. Los motores se mantienen en frenado activo para contrarrestar el torque inducido por la gravedad en las articulaciones.
- **MODE_FAST_MOV:** Modo de desplazamiento rápido orientado a operaciones de posicionamiento manual (*Jogging*). Permite movimientos precisos basados en los deltas de pasos enviados desde la interfaz.
- **MODE_FOLLOW_ROUTINE:** Modo de ejecución de trayectorias. El sistema procesa vectores de PWM y tiempos de ejecución provenientes de archivos de datos, utilizando temporizadores internos para asegurar la detención precisa del movimiento.



4.9. Adquisición de Datos y Gestión de Periféricos I/O

La interacción con el brazo Mitsubishi Move Master EX requiere la gestión precisa de tres tipos de señales críticas: la retroalimentación de posición (encoders), la detección de límites físicos (finales de carrera) y el control de estabilidad (frenos electromecánicos).

4.9.1. Retroalimentación de Posición vía Encoders

A diferencia de los sistemas basados en interrupciones externas que pueden saturar el procesador a altas velocidades, se implementó una estrategia de **muestreo periódico**. Los estados de las fases A y B de cada encoder son leídos cada cierto tiempo dentro de una interrupción de temporizador (*Timer Interrupt*).

Esta técnica permite filtrar el ruido electrónico y asegura que el conteo de pasos sea determinista, evitando que el microcontrolador pierda ciclos de ejecución críticos para la comunicación serial. Los valores acumulados se almacenan en variables de tipo `int32_t` dentro de la estructura `Motor_t` para su posterior envío a LabVIEW.

4.9.2. Detección de Límites (Finales de Carrera)

El robot cuenta con interruptores de límite en configuración Siempre Abierta (NO). Para mejorar la experiencia de usuario y la interpretación de datos en la interfaz de LabVIEW, se implementó una inversión lógica en el firmware.

Aunque el hardware detecta una transición física, el código procesa esta señal para **retornar un valor booleano TRUE** únicamente cuando el sensor está siendo presionado. Esta abstracción facilita la lógica de seguridad en la interfaz, permitiendo que el operario identifique visualmente el estado de colisión o límite de manera intuitiva y directa.

4.9.3. Control de Frenos Electromecánicos

Los frenos del robot son dispositivos de seguridad que evitan el colapso de las articulaciones por gravedad cuando los motores no están energizados. Su estado se gestiona mediante lecturas digitales directas (`digitalRead`), las cuales son integradas en la trama de telemetría. Esta adquisición permite que LabVIEW confirme en tiempo real si el brazo se encuentra en estado de liberación para movimiento o bajo retención de seguridad.

Listing 4.5: Lógica de lectura de señales digitales y acondicionamiento

```
1 // Lectura de finales de carrera con logica positiva para la interfaz
2 uint8_t leerFinalesCarrera() {
3     uint8_t estadoByte = 0;
```

```

4   for (int i = 0; i < NUM_FINAL_C; i++) {
5       // Se retorna TRUE si el sensor esta presionado
6       if (digitalRead(PIN_FINAL_CARRERA[i]) == HIGH) {
7           estadoByte |= (1 << i);
8       }
9   }
10  return estadoByte;
11  }
12
13  // Monitoreo de estado de frenos
14  void monitorearFrenos() {
15      bool estadoFreno1 = digitalRead(PIN_FRENO[0]);
16      bool estadoFreno2 = digitalRead(PIN_FRENO[1]);
17      // Logica de seguridad integrada...
18  }

```

4.9.4. Procesamiento de Encoders y Optimización por Hardware

Para la lectura de los encoders, se descartó el uso de interrupciones externas convencionales (*External Interrupts*), ya que la alta frecuencia de pulsos generada por los cinco motores simultáneamente saturaba el procesador, provocando un comportamiento errático en el giro de los motores y pérdida de pasos.

En su lugar, se implementó una estrategia de **muestreo por interrupción de temporizador (Timer Interrupt Handling)** utilizando el Timer 5 del ATmega2560.

Configuración del Timer 5 y Resolución de Pasos

El Timer 5 se configuró en modo CTC (*Clear Timer on Compare Match*) con un valor de comparación de 199 y un prescaler de 8. Esto genera una frecuencia de muestreo de 10 kHz (cada 100 μ s). Esta frecuencia es lo suficientemente alta para capturar todos los cambios de fase sin sobrecargar la CPU, permitiendo que el hilo principal del programa ejecute la lógica de control y comunicación sin retardos.

En cuanto a la resolución, se implementó una decodificación de tipo **Half-Quad**. Aunque el disco físico del encoder del Mitsubishi Move Master EX posee 200 ranuras, la lógica de

software permite detectar cambios de estado específicos que duplican la resolución efectiva a 400 pasos por revolución, logrando un equilibrio ideal entre precisión posicional y carga computacional.

Algoritmo de Tabla de Búsqueda (Look-up Table)

Para determinar el sentido de giro y el conteo de pasos de forma ultra rápida dentro de la ISR (*Interrupt Service Routine*), se utilizó una tabla de estados precalculada:

Listing 4.6: Tabla de estados para decodificación de Encoders

```
1  const int8_t encoderTable[16] = {  
2      0, 0, 0, 0,    // Sin cambio o estados invalidos  
3      1, 0, 0, -1,   // Incremento detectado / Error  
4      -1, 0, 0, 1,   // Decremento detectado / Error  
5      0, 0, 0, 0     // Sin cambio o estados invalidos  
6  };
```

La lógica consiste en concatenar el estado anterior de las fases A y B con el estado actual, formando un índice de 4 bits (valores de 0 a 15). Este índice accede directamente a la tabla, la cual devuelve un **1** para incrementar, un **-1** para decrementar o un **0** si no hubo cambio o si el cambio detectado es físicamente imposible (ruido).

Pseudocódigo de la Lógica de Lectura

El siguiente pseudocódigo ilustra la operación realizada cada 100 microsegundos:

Algoritmo de Muestreo (ISR Timer 5)

1. **Para cada** motor i del 0 al 4:
 - a) Leer estados físicos de los pines Fase A y Fase B.
 - b) Combinar estados: $Actual = (FaseA \ll 1) \text{ OR } FaseB$.
 - c) Crear índice: $Indice = (Anterior[i] \ll 2) \text{ OR } Actual$.
 - d) Consultar tabla: $Delta = encoderTable[Indice]$.
 - e) Actualizar posición: $Pasos[i] = Pasos[i] + Delta$.
 - f) Guardar estado para el siguiente ciclo: $Anterior[i] = Actual$.
2. **Fin del ciclo**

4.10. Interfaz de Usuario en LabVIEW

La estación de control fue desarrollada en LabVIEW, aprovechando su capacidad para la gestión de datos mediante una arquitectura de manejador de mensajes encolados (QMH). La interfaz se organiza en un panel principal con una barra de navegación lateral que permite conmutar entre tres pestañas operativas: **Dashboard**, **Control Manual** y **Control PID**.

4.10.1. Panel de Monitoreo (Dashboard)

La pestaña de *Dashboard* funciona como el centro de diagnóstico principal. En esta vista se presentan indicadores visuales de tipo carátula para cada una de las articulaciones (Cadera, Hombro, Codo, Antebrazo y Muñeca). Para cada eje, el sistema despliega en tiempo real:

- **Conteo de Pasos:** Valor absoluto del encoder tras el procesamiento en el microcontrolador.
- **Velocidad:** Cálculo derivado de la variación de pasos por unidad de tiempo.
- **Estado de Alarmas:** Indicadores de seguridad para los finales de carrera y estados de error.

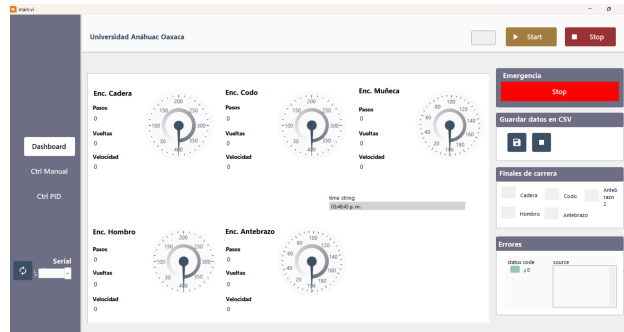


Figura 4.2: Ventana de lectura de los encoders.

4.10.2. Modos de Control: Manual y Trayectorias

La pestaña de **Control Manual** ofrece al operario dos métodos para manipular el brazo:

1. **Control por PWM:** Ajuste directo del ciclo de trabajo para cada motor mediante controles deslizantes (*sliders*), permitiendo movimientos finos para el posicionamiento inicial del robot.
2. **Rutina desde Archivo:** Funcionalidad avanzada que permite cargar un archivo externo (CSV o Excel) con trayectorias predefinidas. El sistema lee estas coordenadas y las envía de forma secuencial al controlador para su ejecución automática.

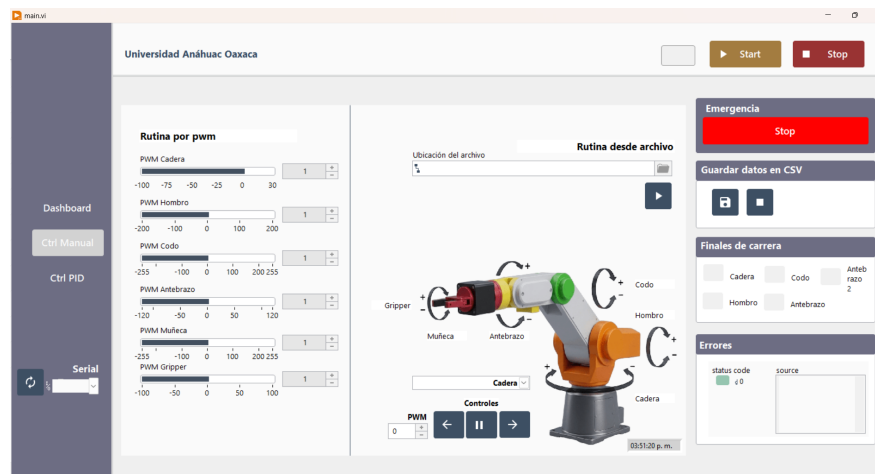


Figura 4.3: Ventana de control de la interfaz.

4.10.3. Sintonización y Control PID

La pestaña de **Control PID** está diseñada para la optimización dinámica del movimiento. Esta sección permite al usuario ajustar las constantes de control (K_p , T_i , T_d) de manera individual para cada articulación. Incluye una gráfica de tiempo real (*Waveform Chart*) que superpone el **Setpoint** (posición deseada) frente a la **Posición Actual**, permitiendo visualizar el error, el sobreimpulso y el tiempo de asentamiento del sistema bajo diferentes condiciones de carga.

Nota de Seguridad: Independientemente de la pestaña activa, la interfaz mantiene un botón de **Paro de Emergencia** global que, al activarse, envía una señal prioritaria para frenar todos los motores y bloquear las articulaciones.

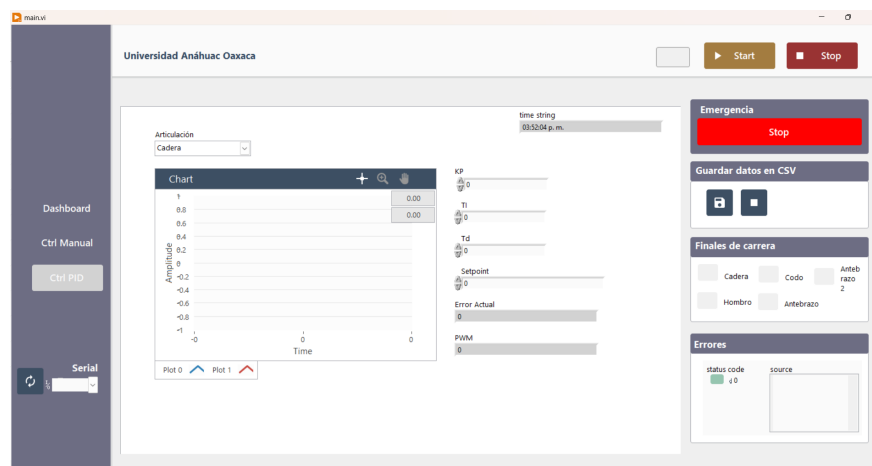


Figura 4.4: Ventana de PID de la interfaz.

Resultados y Validación de Fase

- **Comunicación:** Tasa de transferencia de 500,000 baudios con pérdida de datos nula.
- **Precisión:** Ciclo de control estable a 4ms para la actualización de lazo abierto y cerrado.
- **Seguridad:** Respuesta de detención inmediata ante señales de finales de carrera integrada en la máquina de estados.

Bibliografía

- [1] Allegro MicroSystems. (2024). *ACS712: Fully Integrated, Hall Effect-Based Linear Current Sensor IC with 2.4 kVRMS Isolation and a Low-Resistance Current Conductor [Datasheet]*. Worcester, MA: Allegro MicroSystems, LLC.
- [2] Infineon Technologies AG. (2004). *BTS 7960: High Current PN Half Bridge NovalithIC™ [Datasheet]*. Recuperado de <https://www.infineon.com>
- [3] Hughes, A., and Drury, B. (2019). *Electric motors and drives: fundamentals, types and applications*. Newnes.
- [4] Llamas, L. (2019, 28 octubre). *Controla motores de gran potencia con Arduino y BTS7960*. Luis Llamas. <https://www.luisllamas.es/controla-motores-de-gran-potencia-con-arduino-y-bts7960/>
- [5] Mitsubishi Electric Corporation. (1990). *Movemaster EX: Industrial Robot User Manual*. Nagoya Works, Japan.
- [6] Sebastián, C. M. J., y Sebastián, M. C. J. (2019). *Desarrollo de un sistema de control para el seguimiento de trayectorias en el espacio xyz del robot Mitsubishi Movemaster RV-M1*. Bogotá, D.C.
- [7] Schweizer, T. (2020). *Reviving and Reverse Engineering an Old Robot Arm*. Hackaday.io. Recuperado de: <https://hackaday.io/project/171597-reviving-and-reverse-engineering-an-old-robot-arm>

- [8] Tutorial sensor de corriente ACS712. (s. f.). Naylamp Mechatronics - Perú. https://naylampmechatronics.com/blog/48_tutorial-sensor-de-corriente-ac712.html